MASTER SYLLABUS · VERSION 1.1 (AS BUILT) · 12 OCTOBER 2026

# Agentic AI Engineering

From zero to production-grade, governed AI agents — a 30-week programme with an India-first lens

Prepared for Arunkumar Maniam Rajan · Chennai, India (IST)

Version 1.1 updates the plan to what was actually built and run (sections 11–13)

Tool versions verified on PyPI and official docs on 11 Oct 2026

Reference benchmark: IITM Pravartak × Emeritus Professional Certificate Programme in Agentic AI and Applications

# Contents

**1. Programme overview** ..... **3**

1.1 Design principles ..... 3

**2. Audience and prerequisites** ..... **4**

2.1 Intended learner ..... 4

2.2 Prerequisites ..... 4

**3. How this programme improves on the reference** ..... **4**

**4. Module list** ..... **5**

**5. Week-by-week schedule** ..... **7**

**6. Learning outcomes per module** ..... **8**

**7. Assessment plan** ..... **11**

7.1 Capstone rubric (all capstones) ..... 11

7.2 Capstone briefs ..... 11

**8. Tools and setup** ..... **12**

8.1 Software stack (versions verified 11 Oct 2026) ..... 12

8.2 Hardware and accounts ..... 13

8.3 Repository layout ..... 13

**9. Learning approach and policies** ..... **13**

**10. Reference programme and sources** ..... **14**

**11. As-built deliverables** ..... **15**

**12. What ran on what backend** ..... **16**

**13. Known limitations** ..... **17**

**Appendix A. Week 0 — Python bridge for Kotlin/Java developers (optional)** ..... **18**

**Appendix B. Glossary** ..... **18**

# 1. Programme overview

**Agentic AI Engineering** is a 30-week, project-driven programme that takes an experienced software engineer from first principles of large language models (LLMs) to designing, evaluating, securing and operating multi-agent systems in production. It is organised in five phases that deliberately mirror — and extend — the five pillars of the IITM Pravartak × Emeritus *Professional Certificate Programme in Agentic AI and Applications* used as the reference benchmark (a 5-month core plus a 6-week specialisation).

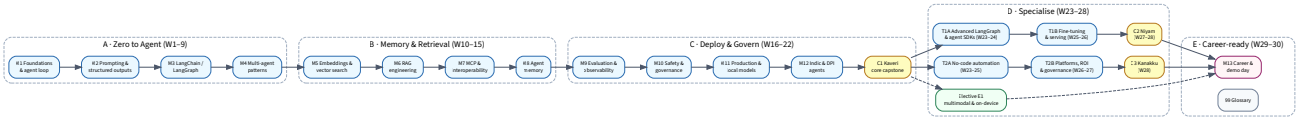



**Figure 1.** Programme map: five phases, thirteen modules, three capstones (C1 core; C2 for Track 1 or C3 for Track 2), two specialisation tracks split into parts A and B, and the optional elective E1.

> **Version 1.1: as built**
>
> Version 1.0 of this syllabus was the plan. Version 1.1, dated 12 October 2026, records what was actually built: 23 PDFs (section 11), which backend each lab ran on (section 12), and the known limitations (section 13). The capstones are named **C1** (core, Kaveri), **C2** (Track 1, Niyam) and **C3** (Track 2, Kanakku). Each track is delivered in two parts: **T1A/T1B** and **T2A/T2B**. Where the plan and the build differ, sections 11–13 take precedence.

| | |
|---|---|
| **Duration** | 30 weeks: 22-week core (≈5 months) + 6-week specialisation + 2-week career sprint |
| **Effort** | 10–12 hours/week (≈ 330 hours total): ~2 h reading, ~1 h quiz/review, 6–8 h labs & projects, 1 h discussion |
| **Format** | Self-paced textbook-style module PDFs, runnable lab repositories, weekly checkpoints, two capstones per learner (C1 plus C2 or C3) |
| **Language** | Python 3.12/3.13 (labs tested on 3.13.5); optional Kotlin/Android elective |
| **Cost to run** | Every lab runs offline on a laptop (mock LLM or local small model). A cloud API key is optional, and none was used to build the course |
| **Outcome** | A portfolio of three production-quality agent repositories, an evaluation & safety dossier, and interview readiness |

## 1.1 Design principles

1. **First principles before frameworks.** You build an agent loop by hand (Module 1) before using LangChain, LangGraph, CrewAI or vendor SDKs, so frameworks become conveniences rather than magic.
2. **Evaluation is a first-class skill.** Every module from M1 ships a small evaluation harness; M9 makes it rigorous.
3. **Safety by construction.** Approval gates for side-effecting tools appear in Module 1; prompt-injection defence, least privilege and governance are graded deliverables, not slides.
4. **Framework-plural, concept-stable.** Concepts (state, tools, memory, handoffs, retrieval, evaluation) are taught once and mapped onto several frameworks, because the ecosystem moves monthly (e.g., AutoGen entered maintenance mode and Microsoft Agent Framework became its successor).

5. **Runs without API keys.** Mock LLMs and local small models (Ollama, Hugging Face on CPU) make every lab reproducible, cheap and private; cloud models are a drop-in upgrade.
6. **India-first context.** Indic languages, token-cost effects of scripts, UPI/ONDC/Account-Aggregator-style workflows (against mocks and sandboxes only), DPDP Act 2023 compliance and INR cost models are woven throughout.
7. **Built for this learner.** Analogies and an elective draw on Android and multimedia engineering (coroutines, lifecycles, streaming pipelines, on-device inference).

# 2. Audience and prerequisites

## 2.1 Intended learner

Professional software engineers (3+ years) who want to become AI/agent engineers — in particular mobile and multimedia engineers who already think in terms of state machines, asynchronous pipelines, performance budgets and device constraints. Track 2 additionally serves tech leads and architects who will own agent strategy.

## 2.2 Prerequisites

| Area | Required | Helpful (taught as needed) |
|---|---|---|
| Programming | Comfortable in one typed language (Kotlin/Java/C++); basic Python syntax | Python typing, asyncio, packaging (taught in M1) |
| Maths | High-school algebra; vectors & matrices; basic probability | Softmax, dot products, entropy (taught in M1) |
| Tools | Git, a terminal, HTTP/JSON APIs | Docker (taught in M11) |
| Hardware | Any 64-bit laptop with 8 GB RAM (16 GB recommended) | A GPU is **not** required; Track 1 fine-tuning uses free/low-cost cloud GPUs |

**Self-check before Week 1:** write a Python function that fetches JSON from a URL and validates two fields; explain what a matrix multiplication of shapes (4×8)·(8×3) produces. If either is hard, do the optional *Week 0 Python bridge* (Appendix A).

# 3. How this programme improves on the reference

The reference programme's syllabus is strong on breadth. This design keeps all of its topics and adds the following:

| Gap / opportunity in the reference outline | What this programme adds | Where |
|---|---|---|
| Frameworks introduced alongside fundamentals | Hand-built agent loop, typed client layer and unit tests before any framework | M1 |
| Structured outputs & tool calling implicit | A dedicated module: JSON Schema, Pydantic contracts, strict/native tool calling, constrained decoding, validation-retry | M2 |

| Gap / opportunity in the reference outline | What this programme adds | Where |
|---|---|---|
| Evaluation focused on hallucination detection | Full evaluation discipline: golden sets, trajectory evals, LLM-as-judge calibration, pass@k, CI regression gates, OpenTelemetry GenAI conventions | M1 (seed), M9 |
| Cost and latency not explicit | Token economics in INR, prompt/semantic caching, model cascades and routing, TTFT/TPOT budgets, quantisation | M1, M11 |
| Local/small models absent | Ollama, llama.cpp, vLLM, GGUF quantisation, SLM selection, offline labs from day one | M1, M11, T1 |
| Agent safety mostly at the end | Approval gates from M1; red-team suite with measured attack-success rate; dual-LLM/privilege-separation patterns | M1, M10 |
| Generic use cases | Indic NLP (AI4Bharat, Bhashini, Sarvam), Indic token fertility, UPI/ONDC/AA/DigiLocker-style workflows on mocks, DPDP Act | M12 + threaded |
| Framework landscape as of 2025 | Updated to Oct 2026: LangChain 1.x/LangGraph 1.2, Microsoft Agent Framework (AutoGen successor), MCP 2026-07-28 spec with extensions, A2A | M3, M4, M7 |
| Memory treated with retrieval | Separate memory-design module with consent, retention and forgetting policies | M8 |
| No on-device/multimodal path | Elective E1: Gemini Nano / ML Kit GenAI, LiteRT-LM (MediaPipe LLM Inference is now maintenance-only), ExecuTorch, ONNX Runtime, llama.cpp and MLC LLM on Android | E1 |
| Career support generic | Role-mapped portfolio (3 repos), a 140-question interview bank with model answers, worked system-design interviews, Android→AI transition narrative | M13 |

# 4. Module list

| # | Module | Weeks | Phase | One-line description |
|---|---|---|---|---|
| M1 | Foundations: Python for AI Engineering, LLM Fundamentals & the Agent Loop | 1–3 | A | Typed, async, tested Python; transformers from scratch; build a tool-using agent with no framework |
| M2 | Prompting, Structured Outputs & Tool Calling | 4–5 | A | Output contracts with JSON Schema/Pydantic, native vs prompted tools, constrained decoding |
| M3 | Frameworks I: LangChain 1.x & LangGraph | 6–7 | A | Stateful graphs, checkpointing, human-in-the-loop interrupts, streaming |
| M4 | Multi-Agent Systems & Design Patterns | 8–9 | A | CrewAI, Microsoft Agent Framework / AutoGen Studio, AG2; supervisor, handoff, debate, plan-execute |
| M5 | Embeddings & Vector Search | 10–11 | B | ChromaDB & FAISS, ANN index families, hybrid BM25 + vector, cross-encoder re-ranking |
| M6 | RAG Engineering | 12–13 | B | Ingestion, chunking, query rewriting, agentic & graph RAG, citations, RAG evaluation |
| M7 | Model Context Protocol & Interoperability | 14 | B | MCP (spec 2026-07-28) servers/clients, transports, auth, security; A2A |
| M8 | Agent Memory Design | 15 | B | Working/episodic/semantic/procedural memory, write & forget policies, context engineering |
| M9 | Evaluation, Observability & Hallucination Detection | 16–17 | C | LangSmith/Langfuse tracing, eval datasets, LLM-as-judge, CI gates |

| # | Module | Weeks | Phase | One-line description |
|---|---|---|---|---|
| M10 | Agent Safety, Security & AI Governance | 18 | C | OWASP LLM Top 10, prompt-injection defence, guardrails, DPDP Act, EU AI Act, ISO/IEC 42001 |
| M11 | Production Engineering, Cost/Latency & Local Models | 19 | C | FastAPI + Docker, streaming, caching, routing, quantised local models |
| M12 | Indic & Digital-Public-Infrastructure Agents | 20 | C | Voice-first Indic agents; UPI/ONDC/AA-style workflows against sandboxes |
| C1 | Core capstone: Kaveri, a bilingual customer-operations agent | 21–22 | C | Integrates M1–M12 under a graded rubric |
| T1A | Track 1 Part A — Advanced LangGraph and agent SDKs | 23–24 | D | Durable graphs, OpenAI Agents SDK, Google ADK, Microsoft Agent Framework, A2A |
| T1B | Track 1 Part B — Fine-tuning and serving | 25–26 | D | LoRA on CPU (QLoRA documented), evaluation of fine-tunes, vLLM/Kubernetes serving (manifests validated) |
| C2 | Track 1 capstone: Niyam, a multi-agent regulatory-compliance RAG | 27–28 | D | LoRA router, verifier, eval gates, load test |
| T2A | Track 2 Part A — No-code automation and process discovery | 23–25 | D | n8n (self-hosted), Zapier and Make (recipes), opportunity mapping |
| T2B | Track 2 Part B — Enterprise platforms, ROI and governance | 26–27 | D | Platform scorecard, INR ROI with Monte Carlo, DPDP governance pack |
| C3 | Track 2 capstone: Kanakku, invoice-to-pay for an Indian SME | 28 | D | n8n + local model, GST validation, approvals, Tally mock, ROI and governance |
| E1 | Elective — Multimodal & On-device Agents (Android) | parallel | D | Gemini Nano/ML Kit GenAI, LiteRT-LM, ExecuTorch, llama.cpp; small VLMs, ASR; hybrid device–cloud routing |
| M13 | Career & Certification | 29–30 | E | Portfolio, resume, LinkedIn, 140-question interview bank, certification plan, demo day |
| — | Course glossary | — | — | `99-glossary.pdf` : every module's glossary merged, plus acronyms |

# 5. Week-by-week schedule

Each week lists the theory focus, the hands-on lab and the checkpoint. “Q” = module quiz, “MP” = mini-project.

| Wk | Module | Theory focus | Lab / build | Checkpoint |
|---|---|---|---|---|
| 0 | (optional) | Python bridge for Kotlin/Java developers | Kotlin→Python kata set | Self-check |
| 1 | M1 | What agents are; Python for AI engineering (uv, typing, Pydantic, asyncio, httpx, pytest); AI-assisted development | Lab 1.1 env & provider-agnostic client; Lab 1.7 async + retries | Env verified |
| 2 | M1 | Tokens, BPE, embeddings, self-attention, transformer blocks, training pipeline, decoding | Labs 1.2–1.5: BPE, sampling, attention from scratch, tiny LLM on CPU | — |
| 3 | M1 | Calling LLMs, tool calling, the ReAct loop, first evals, approval gates | Lab 1.6 agent loop + mini-eval across mock/local backends | Q1, labs |
| 4 | M2 | Prompt anatomy, few-shot, CoT trade-offs, prompt versioning | Prompt test bench with regression cases | — |
| 5 | M2 | JSON Schema, Pydantic contracts, native/strict tool calling, constrained decoding, instructor, pydantic-ai | GST invoice extractor (EN/TA/HI) | Q2, MP1 |
| 6 | M3 | LangChain 1.x agents & middleware; LangGraph StateGraph, reducers, edges | Port the M1 agent to LangGraph | — |
| 7 | M3 | Checkpointers, interrupts (HITL), streaming, durable execution, Studio | Support agent with approval + persistence | Q3 |
| 8 | M4 | Workflow vs agent patterns; supervisor, hierarchical, handoff/swarm, debate, reflection | CrewAI crew & flow | — |
| 9 | M4 | Microsoft Agent Framework workflows, AutoGen Studio prototyping, AG2; A2A intro | Same task in 3 frameworks; cost/quality report | Q4, MP2 |
| 10 | M5 | Embedding models (incl. multilingual/Indic), metrics, exact vs ANN | FAISS Flat/IVF/HNSW benchmark | — |
| 11 | M5 | ChromaDB, metadata filters, BM25, hybrid fusion (RRF), cross-encoder re-ranking | Bilingual hybrid search + rerank | Q5 |
| 12 | M6 | RAG architecture, parsing, chunking strategies, contextual retrieval | Chunking ablation study | — |
| 13 | M6 | Query rewriting, HyDE, agentic/corrective RAG, GraphRAG, citations, ragas/deepeval | Government-circulars RAG with citations | Q6, MP3 |
| 14 | M7 | MCP architecture, primitives, transports, auth, extensions, security; A2A | MCP server for a mock ONDC catalogue | Q7 |
| 15 | M8 | Memory taxonomy, stores, write/forget policies, consent, context engineering | Memory-enabled finance assistant | Q8 |
| 16 | M9 | Eval taxonomy, golden sets, metrics, LLM-as-judge, trajectory evals | 100-item eval set for the RAG agent | — |
| 17 | M9 | Tracing (LangSmith, Langfuse, OpenTelemetry GenAI), dashboards, CI gates, hallucination detection | Instrumented agent + CI regression gate | Q9 |

| Wk | Mod-ule | Theory focus | Lab / build | Checkpoint |
|---|---|---|---|---|
| 18 | M10 | OWASP LLM Top 10, prompt injection, guardrails, sandbox-ing, DPDP/EU AI Act/NIST/ISO 42001 | Red-team & defend; measure attack success rate | Q10 |
| 19 | M11 | FastAPI, Docker, streaming, queues; cost/latency engineer-ing; local & quantised models | Containerised agent with model cas-cade & ₹ cost report | Q11 |
| 20 | M12 | Indic NLP stack, ASR/TTS, code-mixing; UPI/ONDC/AA/Digi-Locker patterns | Tamil voice-to-action agent on mocks | Q12 |
| 21 | C1 | Capstone design review: architecture, threat model, eval plan | Build sprint 1 | Design doc |
| 22 | C1 | Hardening, evaluation, documentation | Build sprint 2 + demo video | **Core capstone** |
| 23 | T1A / T2A | T1: advanced LangGraph · T2: n8n, Zapier, Make | Track lab 1 | — |
| 24 | T1A / T2A | T1: OpenAI Agents SDK & Google ADK · T2: workflows that survive failure, approvals | Track lab 2 | T1 quiz A |
| 25 | T1B / T2A | T1: LoRA/QLoRA with PEFT+TRL · T2: process discovery & opportunity mapping | Track lab 3 | T2 quiz A |
| 26 | T1B / T2B | T1: serving & scaling (vLLM, K8s) · T2: enterprise platforms; ROI & business case in INR | Track lab 4 | T1 quiz B |
| 27 | C2 / T2B | T1: C2 Niyam build · T2: governance operating model | C2 sprint 1 · T2B lab | Design review · T2 quiz B |
| 28 | C2 / C3 | Specialisation capstone (C2 hardening, or the C3 Kanakku week) | Capstone sprint | **Specialisation capstone** |
| 29 | M13 | Portfolio, resume, LinkedIn | Portfolio repos polished | Peer review |
| 30 | M13 | Interview prep: concepts, agent system design, take-home | Mock interviews + demo day | **Final defence, completion** |

# 6. Learning outcomes per module

Outcomes are written to be observable and assessable. On completing each module you will be able to:

**M1 — Foundations.** (1) set up a reproducible, pinned Python environment and structure an AI project with typed models, async I/O, retries and tests; (2) explain tokenization, embeddings, self-attention, the de-coder-only transformer, KV caching and the pre-training → SFT → preference-optimisation pipeline; (3) im-plement softmax, temperature, top-k and top-p sampling and predict their effect; (4) call LLMs through a provider-agnostic client (mock, local, cloud); (5) build, trace and unit-test a tool-using agent loop with an approval gate; (6) quantify how script choice affects token counts and therefore cost for Indic languages.

**M2 — Structured outputs & tool calling.** (1) design output contracts with JSON Schema and Pydantic; (2) choose between native, strict and prompted tool calling and constrained decoding; (3) design tools with good names, granularity, idempotency and error messages; (4) build validation-retry loops that reach ≥95% schema validity.

**M3 — LangChain & LangGraph.** (1) model an agent as a typed state graph with reducers and conditional edges; (2) persist and resume conversations with checkpointers; (3) add human-in-the-loop interrupts; (4) stream tokens and intermediate steps; (5) decide when a framework adds value over hand-written code.

**M4 — Multi-agent systems.** (1) select an appropriate pattern (chain, router, orchestrator-workers, evaluator-optimiser, supervisor, handoff, debate) for a problem; (2) implement it in CrewAI, LangGraph and Microsoft Agent Framework; (3) measure the cost, latency and quality trade-offs of adding agents; (4) explain A2A's role.

**M5 — Embeddings & vector search.** (1) choose and evaluate embedding models including multilingual ones; (2) build and tune FAISS (Flat/IVF/PQ/HNSW) and ChromaDB indexes; (3) implement hybrid search with RRF and cross-encoder re-ranking; (4) report recall@k, MRR and nDCG.

**M6 — RAG engineering.** (1) design ingestion and chunking for heterogeneous documents incl. Indic scripts; (2) implement query rewriting, multi-query, HyDE and agentic/corrective RAG; (3) enforce citations; (4) evaluate faithfulness and context precision/recall and run ablations.

**M7 — MCP & interoperability.** (1) explain MCP's host/client/server model, primitives and transports per the 2026-07-28 specification; (2) build and secure an MCP server with the Python SDK/FastMCP; (3) consume it from two agent frameworks; (4) identify tool-poisoning and confused-deputy risks.

**M8 — Agent memory.** (1) classify memory types and select stores; (2) design write, retrieval and forgetting policies; (3) implement consent-aware long-term memory; (4) apply context-engineering techniques (compaction, scratchpads, sub-agents).

**M9 — Evaluation & observability.** (1) build golden datasets and trajectory evals; (2) calibrate LLM-as-judge against human labels; (3) instrument agents with tracing (LangSmith/Langfuse/OpenTelemetry); (4) detect hallucinations with entailment and citation checks; (5) gate releases in CI.

**M10 — Safety, security & governance.** (1) threat-model an agent with the OWASP LLM Top 10; (2) implement layered prompt-injection defences and measure attack success rate; (3) configure guardrails and PII redaction for Indian identifiers; (4) map obligations under the DPDP Act 2023, EU AI Act and NIST AI RMF; (5) write an agent card and incident runbook.

**M11 — Production, cost & local models.** (1) ship an agent as a FastAPI + Docker service with streaming; (2) build a cost model in INR and reduce it with caching, routing and cascades; (3) meet TTFT/TPOT budgets; (4) select, quantise and serve small local models with Ollama/llama.cpp/vLLM.

**M12 — Indic & DPI agents.** (1) assemble an Indic voice pipeline (ASR → LLM → TTS) with translation/transliteration; (2) design agents that assist UPI/ONDC/AA-style flows without bypassing regulated consent steps; (3) evaluate multilingual quality and fairness; (4) design for low bandwidth and accessibility.

**C1 — Core capstone.** Integrate tools, MCP, RAG, memory, approval gates, tracing, evaluation and security into one deployable bilingual agent, with documented trade-offs.

**T1 — Advanced engineering.** (1) build durable multi-agent graphs; (2) compare OpenAI Agents SDK, Google ADK and Microsoft Agent Framework on one problem; (3) fine-tune a small model with LoRA/QLoRA for tool routing and evaluate it; (4) serve and load-test at scale.

**T2 — Strategy & enterprise workflows.** (1) build agentic automations in n8n, Zapier and Make; (2) evaluate enterprise agent platforms for fit, lock-in and data residency; (3) build an ROI model in INR; (4) design an AI governance operating model and change plan.

**E1 — Multimodal & on-device.** (1) choose among Gemini Nano (ML Kit GenAI), LiteRT-LM, ExecuTorch, ONNX Runtime and llama.cpp for an Android feature; (2) measure tokens/s and memory against a budget; (3) make small-VLM and ASR output safe with validators; (4) design hybrid on-device/cloud routing within privacy, latency, memory and battery budgets.

**M13 — Career.** (1) present three portfolio repositories with eval reports and demos; (2) answer conceptual and agent system-design interview questions; (3) articulate a credible Android→AI-engineer transition story; (4) write resume bullets traceable to logged results; (5) plan the next 90 days against a skills matrix.

# 7. Assessment plan

| Component | Count | Weight | Notes |
|---|---|---|---|
| Labs | 12 module lab sets (M1–M12) | 30% (2.5% each) | Auto-graded tests + short reflection; must include run log |
| Quizzes | 12 (M1–M12) | 15% (1.25% each) | 20 questions; two attempts; answer key released after deadline |
| Mini-projects | MP1 (M2), MP2 (M4), MP3 (M6) | 15% (5% each) | Graded with a 4-criterion rubric |
| Core capstone (C1) | 1 | 20% | Rubric below; live demo + code review |
| Specialisation capstone | 1 (Track 1 or 2) | 15% | Track-specific rubric |
| Career portfolio & participation | 1 | 5% | Portfolio, resume, mock interview |
| **Total** | | **100%** | Pass ≥ 60% overall **and** ≥ 50% in each capstone **and** the final defence passed; distinction ≥ 85% |

Track quizzes (T1A, T1B, T2A, T2B) and the E1 quiz are formative (0%). The **final defence** (60 minutes: capstone demo, technical questions, an unseen system design) is a pass/fail gate. Certificates: Certificate of Completion (Track 1 or Track 2), with Distinction at ≥ 85%, an Elective E1 endorsement if the E1 project scores ≥ 60/100, and a Statement of Participation otherwise (labs for at least 6 modules submitted). It is a course certificate, not an accredited award. Rules and tests: M13 §13.7 and Lab 13.3.

## 7.1 Capstone rubric (all capstones)

| Criterion | Weight | Excellent (90–100%) looks like |
|---|---|---|
| Functionality | 30% | All required capabilities work end-to-end on the demo scenarios and the hidden test set |
| Engineering quality | 20% | Typed, tested (≥70% coverage on core logic), containerised, clear configuration, pinned dependencies |
| Evaluation | 20% | ≥50-case golden set, task & trajectory metrics, judge calibration, regression gate, honest error analysis |
| Safety & governance | 15% | Threat model, injection suite with attack-success rate, approval gates, PII handling, DPDP mapping |
| Documentation & demo | 15% | Architecture diagram, ADRs, cost/latency budget in INR, ≤ 3-minute demo video |

## 7.2 Capstone briefs

**C1 core capstone — "Kaveri" bilingual operations agent.** A customer-operations agent for an Indian D2C retailer or fintech that answers policy questions (RAG with citations), checks order/payment status (mock UPI/ONDC tools via an MCP server), remembers customer preferences with consent, escalates refunds through human approval, and works in English plus Tamil or Hindi. Delivered as a FastAPI + Docker service with tracing and evaluation reports.

**C2 Track 1 capstone — "Niyam" production multi-agent RAG system.** For example, a regulatory-compliance copilot over public RBI/SEBI circulars: planner, retriever, analyst and verifier agents; a LoRA-fine-tuned small router model; hybrid retrieval with re-ranking; full observability; load test; eval gates.

**C3 Track 2 capstone — "Kanakku" end-to-end enterprise agentic workflow.** For example, invoice-to-pay for an Indian SME: email/WhatsApp intake, GST invoice extraction and validation, three-way match against a mock ERP, approval routing, exception handling by an agent, built with n8n/Make plus code where needed; delivered with an ROI model, risk register and governance pack.

# 8. Tools and setup

## 8.1 Software stack (versions verified 11 Oct 2026)

| Purpose | Tool | Version | Used in |
|---|---|---|---|
| Language | Python | 3.13 (3.12 also fine; some libraries require < 3.14) | all |
| Env & packaging | uv or venv + pip | latest | all |
| Data validation | pydantic | 2.14.0 | M1+ |
| HTTP | httpx | 0.28.1 | M1+ |
| Local models | Ollama | 0.40.3 | M1, M11 |
| Local models (Python) | transformers / torch (CPU) | 5.19.0 / 2.14.1 | M1, T1 |
| Agent framework | langchain / langchain-core | 1.4.9 / 1.7.6 | M3+ |
| Graph orchestration | langgraph | 1.2.20 (M3, M4, M7); 1.2.19 (M6, M8–M11, C1, T1) | M3+ |
| Multi-agent | crewai | 1.15.39 | M4 |
| Multi-agent | agent-framework (Microsoft) | 1.25.0 (M4; T1 text cites 1.21.0) | M4, T1 |
| Multi-agent (legacy/maintenance) | autogen-agentchat / autogenstudio | 0.7.5 / 0.4.2.2 | M4 |
| Multi-agent (community) | ag2 | 1.1.2 | M4 |
| Vendor SDKs | openai-agents / google-adk | 0.23.1 / 2.11.0 | T1 |
| MCP | mcp (Python SDK) / fastmcp; spec revision | 2.3.0 / 4.1.6; 2026-07-28 | M7+ |
| Vector search | chromadb / faiss-cpu | 1.5.9 / 1.15.1 | M5+ |
| Embeddings & re-ranking | sentence-transformers | 6.1.0 | M5+ |
| Structured outputs | instructor / pydantic-ai | 1.17.0 / 2.55.0 | M2 |
| Evaluation | ragas / deepeval | 0.4.3 / 4.2.8 | M6, M9 |
| Observability | langsmith / langfuse | 0.14.7 / 4.17.0 | M9+ |
| Serving | fastapi / uvicorn | 0.143.0 / 0.54.0 | M11+ |
| Fine-tuning | peft / trl | 0.21.2 / 1.15.0 | T1 |
| Gateway (optional) | litellm | 1.105.0 | M11 |
| No-code (Track 2) | n8n, Zapier, Make | SaaS / self-hosted | T2 |

**Version policy**

Versions above were read from PyPI on 11 October 2026. Each module pins its own `requirements.txt` and is re-verified when the module is built. When a framework ships a breaking change mid-course, the module notes the delta rather than silently changing code.

## 8.2 Hardware and accounts

- **Minimum:** 8 GB RAM laptop, 20 GB free disk. **Recommended:** 16 GB RAM. No GPU needed for core modules.
- **Optional accounts:** one cloud LLM provider (any OpenAI-compatible endpoint), LangSmith or self-hosted Langfuse, GitHub, Docker Hub. Track 1's QLoRA and vLLM steps need a GPU (e.g., a single 24 GB card for ~6–10 hours); as built, they are documented and validated but were not run (section 13), and the LoRA fine-tune ran on the CPU.
- **Budget guidance:** with local models for development and a cloud model only for final evaluations, total API spend for the whole programme can be kept under ₹3,000; Track 1 GPU time is the main additional cost.

## 8.3 Repository layout

```
ai-course/
  README.md                   # index of every PDF with page counts (generated)
  00-master-syllabus.pdf … 13-module-13-career.pdf, C1/C2/C3, T1A/T1B/T2A/T2B, E1, 99-glossary.pdf
  course-plan.md              # design contract: specs, template, style rules, version pins
  labs/module-01 … module-13/ # per-module labs, tests, run_all.sh, outputs/logs/, outputs/RUN_LOG.md
  labs/capstone-core|capstone-t1|capstone-t2/   # C1, C2, C3
  labs/track1-a|track1-b|track2-a|track2-b/     # track parts
  labs/elective-e1/           # E1 Python labs and Kotlin samples (not compiled)
  labs/course-glossary/ labs/course-index/      # generators for 99-glossary.pdf, README and section
11
  build/                      # build.py, style.css, diagrams/*.dot, check_pdf.sh
  src/                        # Markdown sources (src/parts/ per chapter)
  research/                   # version snapshots and link-check results
```

# 9. Learning approach and policies

- **Weekly rhythm:** Mon–Tue read theory; Wed–Fri labs; Sat checkpoint (quiz or deliverable); Sun rest/review.
- **AI-assistant policy:** coding assistants are encouraged (M1 teaches a disciplined workflow), but every submission must include tests you understand and a short "what I verified" note. Quizzes are closed-assistant.
- **Reproducibility:** submissions must run with `./run_all.sh mock` on a clean machine.
- **Ethics & law:** never test against real payment, banking or government systems; use mocks or official sandboxes; never commit secrets or personal data.

# 10. Reference programme and sources

- Reference programme: IITM Pravartak × Emeritus, *Professional Certificate Programme in Agentic AI and Applications* — https://iitmpravartak.emeritus.org/professional-certificate-programme-in-agentic-ai-and-applications (the URL `iitmpravartak.emeritus.com` given in the brief returned HTTP 404 on 11 Oct 2026; the `.org` page above is the live programme page).
- AutoGen maintenance-mode notice and Microsoft Agent Framework migration guide — https://github.com/microsoft/autogen, https://learn.microsoft.com/en-us/agent-framework/migration-guide/from-autogen/
- MCP specification, revision 2026-07-28 — https://modelcontextprotocol.io/specification/2026-07-28
- Anthropic, *Building effective agents* — https://www.anthropic.com/engineering/building-effective-agents
- OWASP GenAI Security Project, LLM Top 10 — https://genai.owasp.org/llm-top-10/
- Package versions — PyPI project pages, e.g. https://pypi.org/project/langgraph/1.2.19/ (full list in `research/`).

# 11. As-built deliverables

The course was delivered as 23 PDFs. The table below is generated by `labs/course-index/inventory.py` from `pdfinfo` and the files on disk (12 October 2026), so the page counts are the real ones. The same table opens `README.md`.

| File | Contents | Weeks | Pages | Size | Lab code |
|---|---|---|---|---|---|
| `00-master-syllabus.pdf` | Master syllabus (as built, v1.1) | — | 18 | 174 KB | — |
| `01-module-1.pdf` | M1 Foundations: Python, LLM fundamentals, the agent loop | 1–3 | 63 | 649 KB | `labs/module-01` |
| `02-module-2.pdf` | M2 Prompting, structured outputs and tool calling | 4–5 | 56 | 492 KB | `labs/module-02` |
| `03-module-3.pdf` | M3 LangChain 1.x and LangGraph | 6–7 | 52 | 389 KB | `labs/module-03` |
| `04-module-4.pdf` | M4 Multi-agent systems and design patterns | 8–9 | 44 | 419 KB | `labs/module-04` |
| `05-module-5.pdf` | M5 Embeddings and vector search | 10–11 | 64 | 762 KB | `labs/module-05` |
| `06-module-6.pdf` | M6 RAG engineering | 12–13 | 84 | 744 KB | `labs/module-06` |
| `07-module-7.pdf` | M7 Model Context Protocol and interoperability | 14 | 44 | 336 KB | `labs/module-07` |
| `08-module-8.pdf` | M8 Agent memory design | 15 | 43 | 338 KB | `labs/module-08` |
| `09-module-9.pdf` | M9 Evaluation, observability and hallucination detection | 16–17 | 56 | 463 KB | `labs/module-09` |
| `10-module-10.pdf` | M10 Agent safety, security and AI governance | 18 | 52 | 404 KB | `labs/module-10` |
| `11-module-11.pdf` | M11 Production engineering, cost/latency and local models | 19 | 45 | 371 KB | `labs/module-11` |
| `12-module-12.pdf` | M12 Indic and digital-public-infrastructure agents | 20 | 50 | 395 KB | `labs/module-12` |
| `C1-core-capstone.pdf` | C1 Core capstone: Kaveri bilingual operations agent | 21–22 | 33 | 282 KB | `labs/capstone-core` |
| `T1A-track1-advanced-engineering.pdf` | T1A Advanced LangGraph and agent SDKs | 23–24 | 51 | 353 KB | `labs/track1-a` |
| `T1B-track1-finetuning-serving.pdf` | T1B Fine-tuning (LoRA) and serving | 25–26 | 51 | 377 KB | `labs/track1-b` |
| `C2-T1-capstone.pdf` | C2 Track 1 capstone: Niyam multi-agent regulatory RAG | 27–28 | 37 | 319 KB | `labs/capstone-t1` |
| `T2A-track2-nocode-automation.pdf` | T2A No-code/low-code automation and process discovery | 23–25 | 59 | 433 KB | `labs/track2-a` |

| File | Contents | Weeks | Pages | Size | Lab code |
|---|---|---|---|---|---|
| `T2B-track2-enterprise-roi-governance.pdf` | T2B Enterprise platforms, ROI and governance | 26–27 | 50 | 449 KB | `labs/track2-b` |
| `C3-T2-capstone.pdf` | C3 Track 2 capstone: Kanakku invoice-to-pay | 28 | 37 | 403 KB | `labs/capstone-t2` |
| `E1-multimodal-on-device-android.pdf` | E1 Multimodal and on-device agents on Android (optional) | parallel to 23–28 | 49 | 392 KB | `labs/elective-e1` |
| `13-module-13-career.pdf` | M13 Career and certification | 29–30 | 55 | 348 KB | `labs/module-13` |
| `99-glossary.pdf` | Course glossary and acronyms | — | 20 | 122 KB | `labs/course-glossary` |
| **Total** | 23 PDFs | 30 weeks | **1113** | | |

Changes from version 1.0 of the plan:

- **Capstone naming.** The three capstones are C1 (core, Kaveri), C2 (Track 1, Niyam) and C3 (Track 2, Kanakku). Each learner completes C1 and one of C2 or C3.
- **Track splits.** Each track was too large for one booklet, so each is split into part A and part B. Track 1: T1A (weeks 23–24) and T1B (weeks 25–26), then C2 (weeks 27–28). Track 2: T2A (weeks 23–25) and T2B (weeks 26–27), then C3 (week 28).
- **E1.** The elective is a separate optional PDF that can run alongside either track. Because the Google AI Edge documentation now marks MediaPipe LLM Inference as maintenance-only, E1 teaches LiteRT-LM and ML Kit GenAI instead.
- **Glossary.** Appendix B now points to `99-glossary.pdf`. It has 336 unique terms (merged from 397 module entries) and 69 acronyms.

# 12. What ran on what backend

No cloud LLM API was called anywhere in the course. “Mock” means a deterministic scripted model, so results can be reproduced exactly. “Ollama” means a small Qwen2.5-Instruct model (0.5B, 1.5B or 3B, Q4_K_M) served locally by Ollama 0.40.3. Every run was on one 8-core x86-64 CPU-only VM with 15 GB of RAM and Python 3.13.5. Each module’s Appendix A has the full run log.

| Unit | Backends actually used | Notable items not run |
|---|---|---|
| M1 | Mock; Ollama qwen2.5 0.5B; Hugging Face SmolLM2-360M on CPU | — |
| M2 | Mock; Ollama qwen2.5 1.5B; HF | — |
| M3 | Mock; Ollama qwen2.5 1.5B (LangGraph) | — |
| M4 | Mock; Ollama 1.5B (including CrewAI in its own venv) | — |
| M5 | HF embedding models on CPU; FAISS/HNSW; no generator | — |
| M6 | HF retrievers and rerankers; mock; Ollama 1.5B/3B | some Ollama variants; Ragas/DeepEval (code only) |
| M7 | Mock; a short Ollama 1.5B run; local MCP servers | — |

| Unit | Backends actually used | Notable items not run |
|---|---|---|
| M8 | Deterministic memory labs; Ollama 1.5B (Lab 8.3) | — |
| M9 | Ollama 1.5B; NLI models on CPU; self-hosted Arize Phoenix | hosted tracing services (no accounts) |
| M10 | SusceptibleMock; Ollama 1.5B; offline classifiers; benign canary attacks only | NeMo Guardrails, Guardrails AI, LLM Guard, garak, promptfoo, Prompt Guard 2 (gated) |
| M11 | Mock; Ollama 0.5B/1.5B/3B; rootless podman | podman compose, locust, vLLM (no GPU), speculative decoding |
| M12 | Offline tokenisers (Qwen, GPT-2, Sarvam and others); Ollama 0.5B/1.5B/3B | every real DPI/Indic API, ASR/TTS, IndicTrans2, Indic LLM inference |
| C1 | Mock; Ollama 1.5B | — |
| T1A | Mock; Ollama 1.5B through the OpenAI Agents SDK, Google ADK and LangGraph | hosted model APIs |
| T1B | CPU LoRA fine-tune of Qwen2.5-0.5B (PEFT/TRL); kubeconform/kube-linter manifest checks | QLoRA/bitsandbytes and vLLM (GPU only); a real Kubernetes cluster |
| C2 | HF LoRA router; Ollama 1.5B; podman | kind cluster (failed to create under rootless podman) |
| T2A | n8n 2.42.6 in podman; Ollama 1.5B | Zapier and Make (recipes only, no accounts) |
| T2B | Python ROI model, Monte Carlo, xlsx; pricing read from official pages | every enterprise agent platform (no accounts) |
| C3 | n8n plus Ollama 1.5B; a rules-only reference pipeline | real Tally, bank or e-mail integrations (all mocked) |
| E1 | HF SmolVLM, Whisper and MMS on CPU; Ollama as an on-device proxy | Kotlin samples not compiled; no Android device or emulator; Gemini Nano |
| M13 | Python standard library only (results bank, question bank, grader, portfolio check) | — |
| Glossary | Python standard library only | — |

# 13. Known limitations

1. **No cloud API key.** All LLM results come from mocks or small local models. Quality numbers do not predict how frontier models would score, because nothing was measured against hosted APIs. Prices for hosted models are taken from official pages and marked as illustrative.
2. **GPU-only items were not run.** QLoRA/bitsandbytes, vLLM serving, speculative decoding and a real Kubernetes deployment (kind failed under rootless podman) are documented and their configurations validated, but none of them was executed. The fine-tune in T1B/C2 is a CPU LoRA on a 0.5B model.
3. **Native-speaker review needed.** All Tamil and Hindi text (prompts, test sets, transcripts and examples in M12, C1, E1 and elsewhere) was written without a native-speaker review. Have it reviewed before you rely on it.
4. **Synthetic data and small test sets.** Every dataset is synthetic, and evaluation sets are small (tens of items). Confidence intervals are reported where they matter, and they are wide.

5. **Android.** No device or emulator was available. Kotlin code in E1 is illustrative and was not compiled. On-device throughput was approximated with an Ollama proxy on the VM CPU. E1's memory cap could not be enforced on the box.
6. **Licences.** Some models used in labs carry non-commercial licences (for example MMS-TTS, CC-BY-NC). Check every licence before using a model in a product.
7. **SaaS platforms.** Zapier, Make, hosted tracing and the enterprise agent platforms were not used, because no accounts were created. Their recipes and comparisons come from official documentation.
8. **Portfolio gaps (closed) and two E1 measurement bugs.** Lab 13.4 first found that the capstones lacked demo scripts and that C3 and E1 lacked READMEs, a Dockerfile and (E1) tests. These were closed on 12 October 2026 (M13 §13.3.6). Writing E1's tests exposed two measurement bugs. Re-scored from saved outputs, Whisper-small's Tamil WER is 61.4%, not the 31.7% first printed, and the SmolVLM GSTIN counts are one higher. The scoring code is fixed, and the E1 booklet was rebuilt with the corrected figures and an erratum. The other Tamil figures (tiny, base, noisy audio) cannot be re-scored without re-running the models and are shown as n/a.
9. **Version drift.** Versions and links were verified on 11–12 October 2026 and will drift. Re-verify them ( `research/` ) before relying on them.

# Appendix A. Week 0 — Python bridge for Kotlin/Java developers (optional)

| Kotlin / Android idea | Python equivalent | Covered in |
|---|---|---|
| `data class` | `@dataclass` / Pydantic `BaseModel` | M1 §1.3 |
| Null safety `String?` | `str \| None` + type checker (mypy/pyright) | M1 §1.3 |
| Coroutines, `suspend` , `Dispatchers.IO` | `async def` , `await` , event loop, `asyncio.to_thread` | M1 §1.4 |
| `async { }` + `awaitAll()` | `asyncio.gather` / `TaskGroup` | M1 §1.4 |
| Retrofit + OkHttp interceptors | httpx client + event hooks / tenacity retries | M1 §1.4 |
| Gradle version catalog | `pyproject.toml` + lock file (uv) / pinned `requirements.txt` | M1 §1.2 |
| JUnit | pytest | M1 §1.5 |
| `sealed class` | `Literal` / `Union` types, `match` statement | M1 §1.3 |

# Appendix B. Glossary

The consolidated course glossary is **`99-glossary.pdf`** : every module's glossary merged and de-duplicated, with module cross-references and an acronym list. It is generated by `labs/course-glossary/build_glossary.py` from the module sources and the seed glossary in `course-plan.md` (section 8).